

Herencia, polimorfismo, y sobrecarga

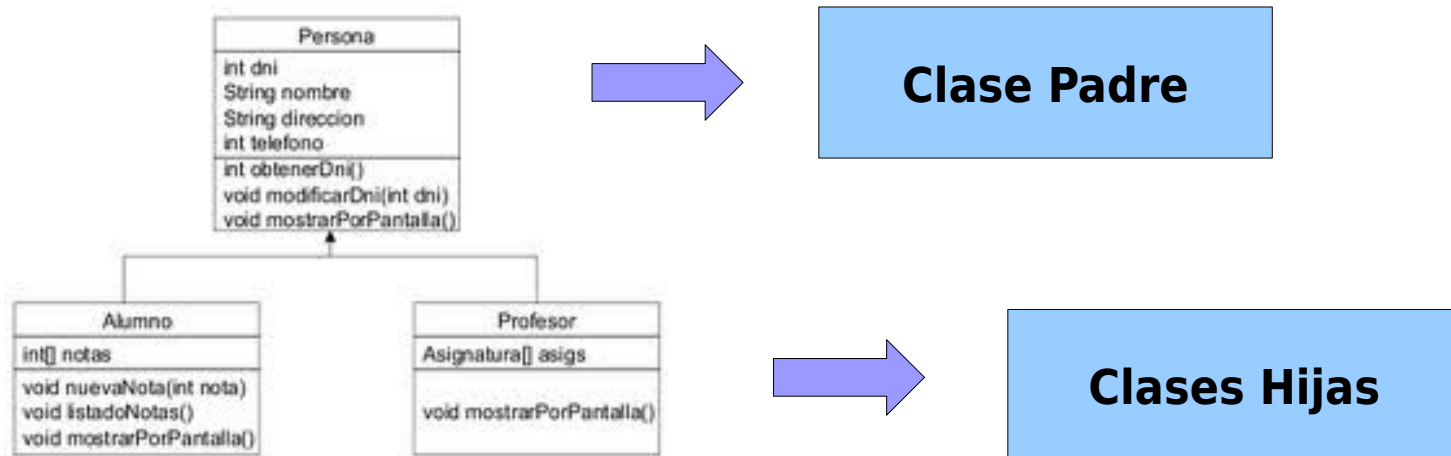
Upcasting y Downcasting de Objetos

Métodos y Clases final

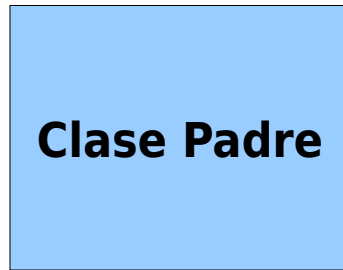
Clases Abstractas e Interfaces

Herencia

- La reusabilidad puede lograrse mediante **herencia**.
- Un comportamiento definido en una *superclase* es heredado por sus *subclases*.
- Las subclases extienden la funcionalidad heredada
- Esto nos permite definir la mayor cantidad de funcionalidades y atributos y luego reutilizarlas.



Herencia



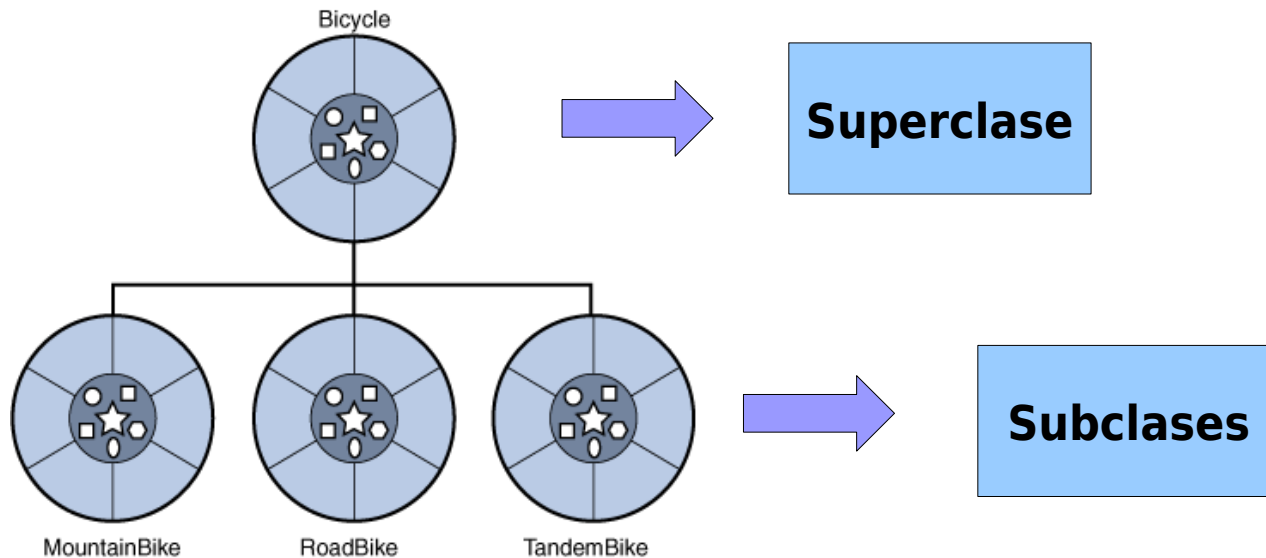
- Definimos atributos y comportamientos comunes.
- Es considerada como un tipo de generalización.



- Reutilizamos todo lo definido en la clase padre.
- Podemos ampliar atributos y comportamientos o *redefinirlos*.

Herencia

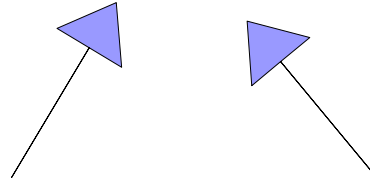
- Una clase hereda de su padre todos los atributos y métodos públicos y protegidos.
- Los constructores no son heredados pero pueden invocarse.
- Los métodos y atributos privados no se heredan.
- Si la subclase esta en el mismo paquete que la clase padre, también se heredan los miembros default.



Herencia en Java

- Para representar la herencia utilizaremos la palabra clave *extends*.
- Java no soporta la herencia múltiple.
- Si no especificamos herencia, todo extiende de la clase *Object*.

```
public class Persona {  
}
```

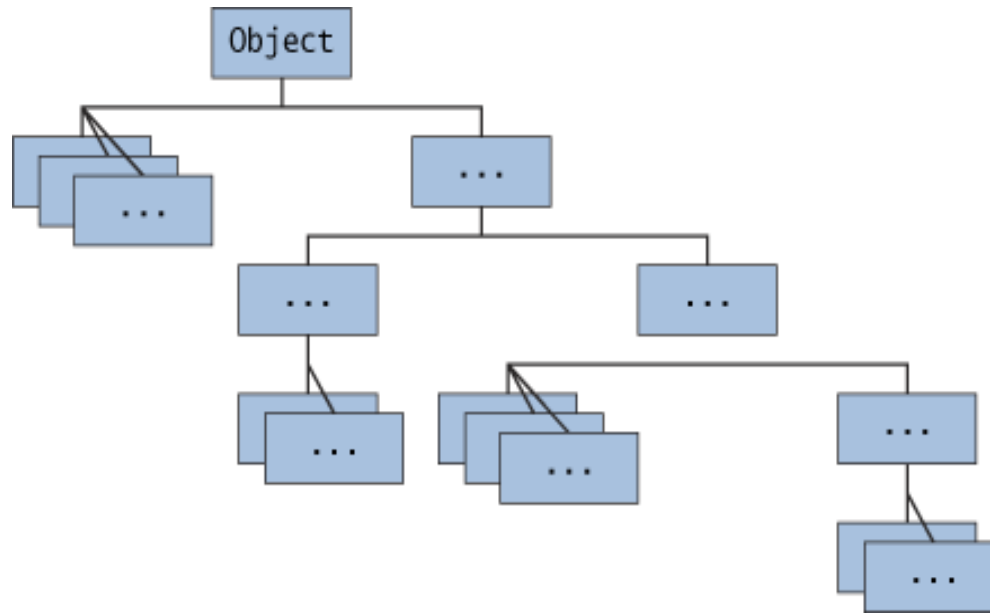


```
public class Profesor extends Persona {  
  
}
```

```
public class Alumno extends Persona {  
  
}
```

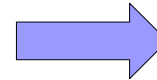
La classe Object

- Es la clase padre de toda clase en Java.
- Toda clase hereda implícitamente de ella.
- Provee métodos muy útiles para sobrescribir.
- Si una clase extiende directamente de ella no es necesario especificarlo



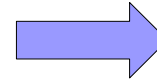
Métodos de la clase Object

```
@Override
protected Object clone() throws CloneNotSupportedException {
    // TODO Auto-generated method stub
    return super.clone();
}
```



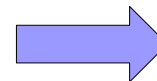
Crear y retornar una copia del objeto

```
@Override
public boolean equals(Object arg0) {
    // TODO Auto-generated method stub
    return super.equals(arg0);
}
```



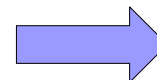
Definir cuando dos objetos son iguales

```
@Override
protected void finalize() throws Throwable {
    // TODO Auto-generated method stub
    super.finalize();
}
```



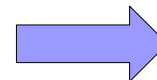
Invocado por el recolector de basura cuando se libera la memoria del objeto

```
@Override
public int hashCode() {
    // TODO Auto-generated method stub
    return super.hashCode();
}
```



Retorna un valor único que identifica a un objeto

```
@Override
public String toString() {
    // TODO Auto-generated method stub
    return super.toString();
}
```



Retorna la representación en String de un objeto

Métodos de la clase Object

Un ejemplo

```
public class Punto {  
    private int x;  
    private int y;  
  
    public Punto(int x, int y) {  
        super();  
        this.x = x;  
        this.y = y;  
    }  
  
    @Override  
    public String toString() {  
        return "X = " + x + " " + "Y = " + y;  
    }  
}
```

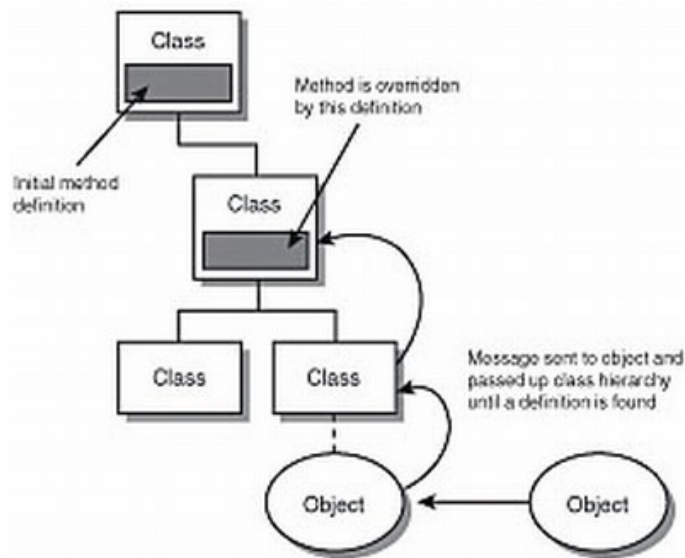
Sobrecargamos el método *toString*

```
public class ToStringDemo {  
    public static void main(String[] args) {  
        Punto p = new Punto(10, 10);  
        System.out.println(p);  
  
        String x = p + " probando";  
        System.out.println(x);  
    }  
}
```

Utilizamos una referencia como String

Sobreescritura de métodos

- Se da en el contexto de relaciones de herencia.
- Consiste en reescribir la implementación de un método de *instancia*, con su mismo nombre y argumentos (*firma del método*).
- Sólo pueden sobreescribirse los métodos no declarados como *final*
- Si una clase hija sobreescribe un método de su clase padre, entonces ocultará al mismo.



Dinámicamente la JVM determina a que método llamar

Sobreescritura de métodos

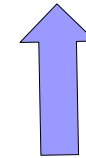
Un ejemplo

```
public class EjemploSobreCarga {  
    public static void main(String[] args) {  
  
        Gato miGato = new Gato();  
        Animal miAnimal = miGato;  
  
        Animal.metodoClase();  
  
        miAnimal.metodoInstancia();  
  
    }  
}
```

Animal.metodoClase()

miAnimal.metodoInstancia()

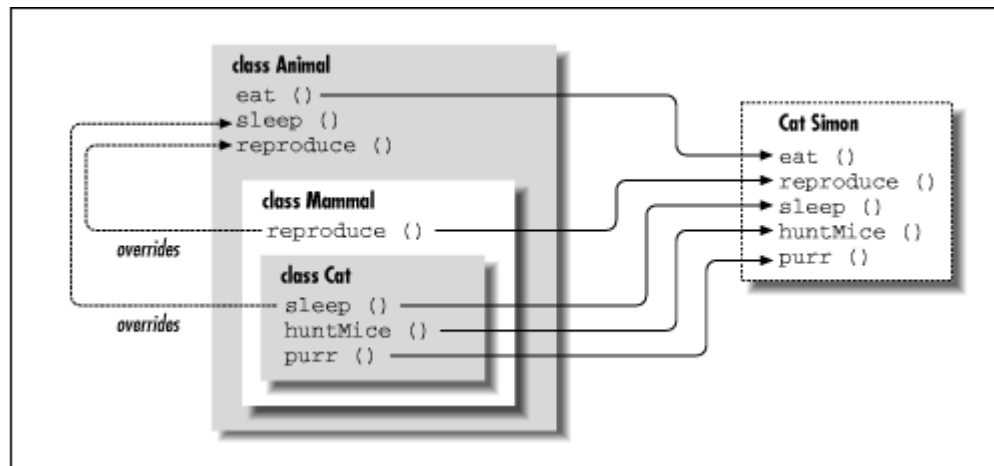
```
public class Animal {  
  
    public static void metodoClase() {  
        System.out.println("Metodo de clase animal");  
    }  
  
    public void metodoInstancia() {  
        System.out.println("Metodo de instancia animal");  
    }  
}
```



```
public class Gato extends Animal {  
  
    public static void metodoClase() {  
        System.out.println("Metodo de clase gato");  
    }  
  
    public void metodoInstancia() {  
        System.out.println("Metodo de instancia gato");  
    }  
}
```

Sobreescritura de métodos

- La lista de argumentos debe ser idéntica en tipos y orden en ambos métodos.
- Los modificadores de acceso no pueden ser más restrictivos en la clase padre que en la subclase.
- Obtendremos un error de compilación si intentamos cambiar un método de instancia en una superclase a método de instancia en la subclase y viceversa.



Sobrecarga de métodos

- Dos o más métodos definidos en la *misma clase* y con igual nombre pueden sobrecargarse.
- Para ello es necesario que tengan el mismo nombre y tipo de retorno, pero *diferentes listas de argumentos (en número o tipos)*
- La JVM en tiempo real determina los parámetros con los cuales estamos invocando a un método y selecciona el correspondiente.

```
public class Artista {  
    public void dibujar(int s) {  
    }  
  
    public void dibujar(String s) {  
    }  
  
    public void dibujar(int s, long y) {  
    }  
}
```

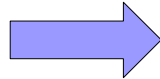


Métodos con el mismo nombre,
pero con distinta firma.

Variable *super*

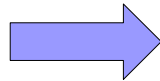
- Utilizada durante la sobreescritura, para acceder a métodos o atributos de instancia de la superclase ocultos para la subclase.
-
- Referencia a miembros de la superclase de la clase dada.

Referencia *this*



Apunta a atributos y métodos de la *clase actual*

Referencia *super*

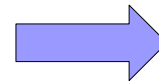


Apunta a atributos y métodos de la *clase padre*

Variable *super*

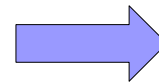
Un ejemplo

```
public class Persona {  
    private String nombre;  
    private String apellido;  
    private Long edad;  
    public void mostrarInfo() {  
        System.out.println("Nombre: " + nombre);  
        System.out.println("Apellido: " + apellido);  
        System.out.println("Edad: " + edad);  
    }  
}
```



Clase padre que define el método *mostrarInfo()*

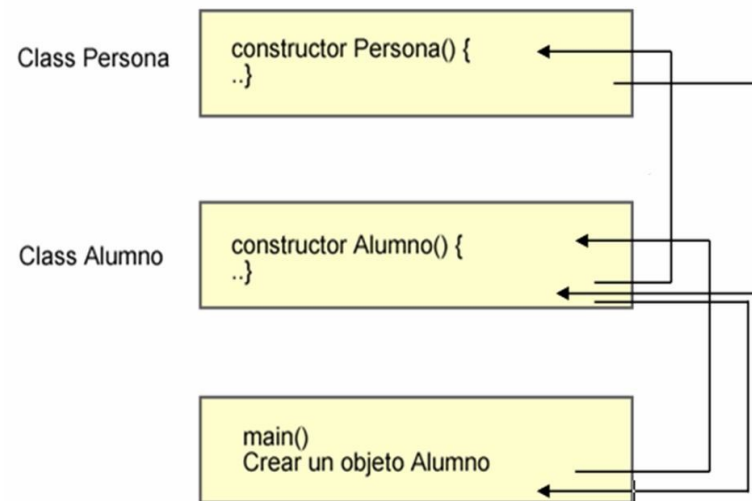
```
public class Artista extends Persona {  
    public void mostrarInfo() {  
        super.mostrarInfo();  
        System.out.println("Profesion: Artista");  
    }  
}
```



Clase que sobrescribe el método e invoca a su definición en la clase padre

Encadenamiento de constructores

- Debido a que los constructores son un tipo especial de métodos, también pueden sobrecargarse.
-
- Esto nos permite asegurar mínimamente como se inicialicen los objetos, no importando que constructor se utilice.



Encadenamiento de constructores

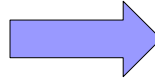
Un ejemplo

```
public Persona(String nombre, String apellido, Long edad) {  
    super();  
    this.nombre = nombre;  
    this.apellido = apellido;  
    this.edad = edad;  
}
```

```
public Persona() {  
    this("", "", 0L);  
}
```

```
public Persona(String nombre) {  
    this(nombre, "", 0L);  
}
```

```
public Persona(String nombre, String apellido) {  
    this(nombre, apellido, 0L);  
}
```



```
Persona p = new Persona();
```

```
Persona p2 = new Persona("Nestor", "Flores", 18L);
```

```
Persona p3 = new Persona("Invitado");
```

```
Persona p4 = new Persona("Alumno", "Del Curso");
```

Aseguramos que mínimamente como se creen los objetos tengan un estado mínimo

Downcasting y Upcasting de Objetos

Un objeto de un tipo se puede tratar como objeto de otro tipo siempre que la clase fuente y la clase destino estén relacionadas por herencia.

-

Upcasting: conversión de clase derivada a la clase base, se hace implícitamente.

```
Alumno a = new Alumno("Juan", "Perez", 18L);
```

```
Persona p = a;
```

Downcasting: conversión de clase base a la clase derivada. Es explícito.

```
Persona pers = new Alumno("Carlos", "Gomez", 17L);
```

```
Alumno al = (Alumno)pers;
```

Alumno es subclase de Persona

El operador *instanceof*

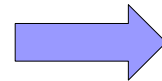
- Verifica que una referencia a un objeto sea de un tipo o de un subtipo dado.
-
- Podemos utilizarlo antes de convertir un objeto para evitar errores

```
Alumno alum = new Alumno("Esteban", "Paz", 35L);  
Profesor prof = (Profesor)alum;
```



Error de compilación

```
if (alum instanceof Persona) {  
    Persona per = (Persona) alum;  
  
}
```



Uso correcto

Alumno no es subclase de Profesor

Métodos y clases *final*

El calificador *final* condiciona el diseño de una jerarquía de herencia.

•

Clases Final:

- No pueden ser extendidas.

```
public final class Persona
```

Métodos Final:

- No pueden ser sobrescritos.
- Los métodos estáticos son automáticamente *final*.

```
public static final void getInfo() {  
  
}
```

Se suele usar para evitar modificar una implementación

Métodos y clases *abstract*

El calificador *abstract* condiciona el diseño de una jerarquía de herencia.

-

Clases Abstract:

- No pueden ser instanciadas.

```
public abstract class Persona
```

Métodos Abstract:

- No pueden tener implementación.
- Deben ser implementados para las clases no abstractas que extiendan de su clase.

```
public abstract void trabajar();
```

Métodos y clases *abstract*

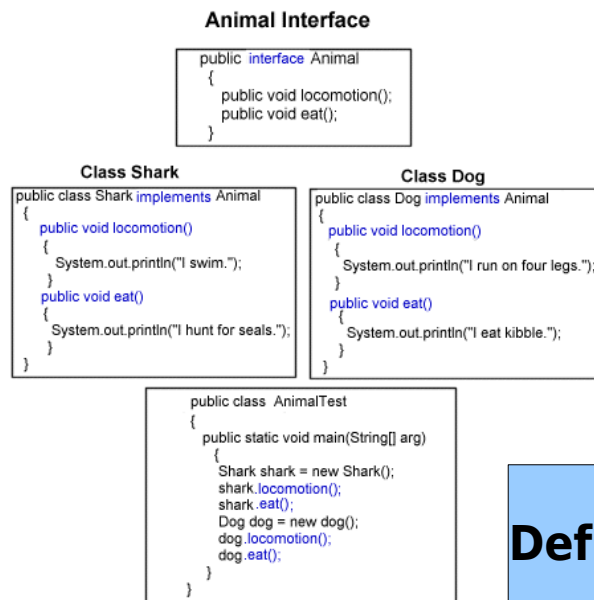
- Cuando un método está presente en todas las clases de una jerarquía pero se implementa en forma diferente conviene definirlo como abstracto.
-
- Una clase con al menos un método abstracto debe declararse como abstracta.
-
- Una clase que extiende de una clase abstracta debe:
 - ✓ Implementar todos sus métodos abstractos o ...
 - ✓ Declararse como abstracta.

```
public class Alumno extends Persona {  
    public void trabajar() {  
        System.out.println("Ir a la escuela");  
    }  
}
```

```
public class Profesor extends Persona {  
    @Override  
    protected void trabajar() {  
        System.out.println("Dar clases");  
    }  
}
```

Interfaces

- Bloques de códigos que poseen declaraciones de métodos y opcionalmente constantes.
-
- Todos sus métodos son abstractos, por ende no se definen.
-
- Una interface puede ser implementada por N clases.
-
- Una una clase implementa una interfaz se compromete a cumplir con su implementación, es decir se firma un contrato entre la clase y la interfaz.



Definimos interfaces utilizando la palabra clave *interface*

Interfaces

- Brindan la declaración de un comportamiento sin brindar la implementación.

```
public interface Imprimible {
```

- ```
 public void imprimir();
```
- ```
}
```

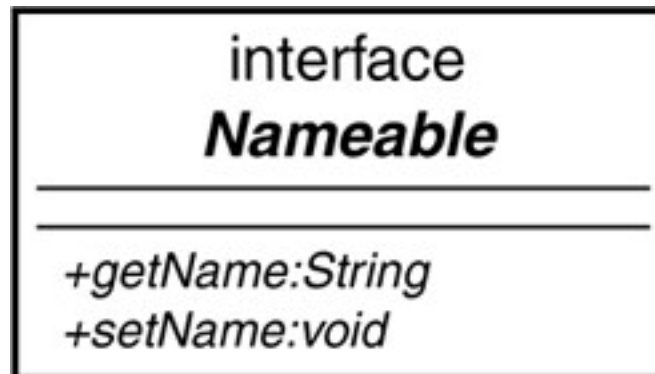
- Cada clase que se compromete a realizar una interfaz debe implementar todos sus métodos

```
public class DocumentoTexto implements Imprimible {
```

```
    public void imprimir() {  
        System.out.println("imprimiendo el documento");  
    }  
}
```

Interfaces

- Una interfaz puede contener:
 - ✓ Declaraciones de métodos públicos.
 - ✓ Constantes: atributos públicos, estáticos y finales.
- Las interfaces no pueden ser instanciadas.
- Si es pública debe coincidir con el nombre del archivo fuente (.java).
- Una referencia a una interfaz puede ser asignada a un objeto de cualquier clase que implemente esa interfaz.



Interfaces

Para tener en cuenta:

La clase **B** extiende de la clase **A** e implementa la interface **I**

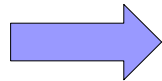
```
public class B extends A implements I {  
}
```

```
B b = new B();
```



Se crea un objeto **B** referenciado por variable **b** de tipo de referencia **B**

```
A a = new B();
```



Se crea un objeto **B** referenciado por variable **a** de tipo de referencia **A**

```
I i = new B();
```

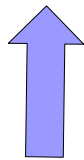


Se crea un objeto **B** referenciado por variable **i** de tipo de referencia **I**

Interfaces

Un ejemplo

```
public class ActionCharacter {  
    public void figth() {  
    }  
}
```



```
public class Hero extends ActionCharacter implements CanFight, CanSwim, CanFly {  
  
    @Override  
    public void fly() {  
        // TODO Auto-generated method stub  
    }  
  
    @Override  
    public void swim() {  
        // TODO Auto-generated method stub  
    }  
}
```

```
public interface CanFight {  
    public void figth();  
}
```

```
public interface CanFly {  
    public void fly();  
}
```

```
public interface CanSwim {  
    public void swim();  
}
```

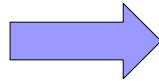
Interfaces

Utilizando todo lo anterior

```
public class Adventure {  
    public static void t(CanFight x) {  
        x.figth();  
    }  
  
    public static void u(CanSwim x) {  
        x.swim();  
    }  
  
    public static void v(CanFly x) {  
        x.fly();  
    }  
  
    public static void w(ActionCharacter x) {  
        x.figth();  
    }  
  
    public static void main(String[] args) {  
        Hero h = new Hero();  
        t(h);  
  
        u(h);  
  
        v(h);  
  
        w(h);  
    }  
}
```



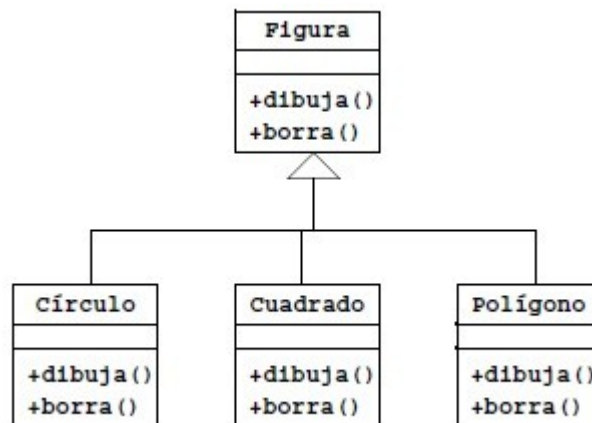
Los métodos reciben como parámetro interfaces



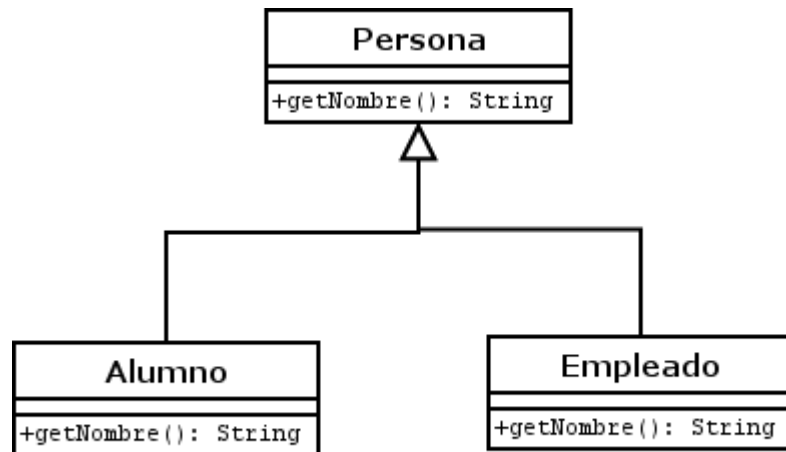
Se pasa un objeto de tipo *Hero* en tiempo de ejecución

Polimorfismo

- Una variable de referencia cambia el comportamiento según el tipo de objeto al que apunta.
-
- Una variable trata a objetos de una clase como objetos de una superclase y se invoca dinámicamente el método correspondiente (*binding dinámico*)
-
- Si tenemos un método que espera como parámetro una variable de clase X, podemos invocarlo usando subclases pasando como parámetros referencias a objetos instancia de subclases de X.



Polimorfismo



```
Persona ref;

Alumno alunObj = new Alumno();

Empleado empObj = new Empleado();

ref = alunObj;    //ref apunta a un objeto de la clase Alumno

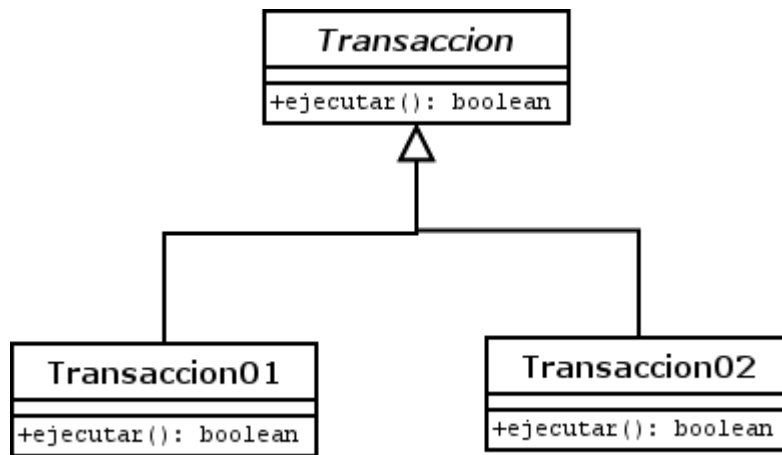
//getNombre() de la clase alumno

String nombre = ref.getNombre();
System.out.println(nombre);

ref = empObj;    //ref apunta a un objeto de la clase Empleado

//getNombre() de la clase empleado
nombre = ref.getNombre();
System.out.println(nombre);
```

Polimorfismo



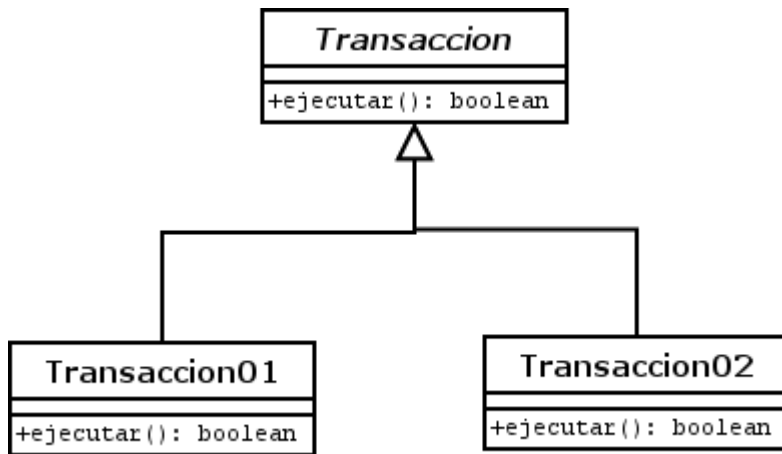
```
public class GestorTransacciones {  
    public static void ejecutar(Transaccion t) {  
        t.ejecutar();  
    }  
}
```

```
public abstract class Transaccion {  
    public abstract boolean ejecutar();  
}
```

```
public class TransaccionBancaria extends Transaccion {  
    @Override  
    public boolean ejecutar() {  
        // Logica del metodo ...  
        System.out.println("Transaccion Bancaria");  
        return false;  
    }  
}
```

```
public class TransaccionOnline extends Transaccion {  
    @Override  
    public boolean ejecutar() {  
        // Logica del metodo ...  
        System.out.println("Transaccion On-Line");  
        return false;  
    }  
}
```

Polimorfismo



```
TransaccionBancaria t1 = new TransaccionBancaria();
TransaccionOnline t2 = new TransaccionOnline();
GestorTransacciones gt = new GestorTransacciones();
gt.ejecutar(t1); //Ejecutar transaccion bancaria
gt.ejecutar(t2); //Ejecutar transaccion online
```

Ejecutamos el método de la clase correspondiente

¿Dudas?



¿Consultas?



Capítulo 2: Java y la Programación Orientada a Objetos

Información de contacto

Web:

<http://www.gugler.com.ar>

<http://cursos.gugler.com.ar>

<http://www.facebook.com/gugler.com.ar>

<http://www.twitter.com/cgugler>

Mail:

contacto@gugler.com.ar

academica@gugler.com.ar

administracion@gugler.com.ar